FreshBasket UAE

Data Structures & Algorithms

Group Assessment — Complete Solutions

Optimizing Grocery Quick-Commerce Operations Across Dubai

Using Data Structures & Algorithms

April 2026 | MAIB Assessment

Layer 1: Worksheet

Problem 1: Build the Network & Explore It

Topics: Graph Representation, BFS, DFS

Part A — Graph Drawing & Adjacency List

The FreshBasket delivery network is an undirected weighted graph with 10 nodes (S1–S5 dark stores, Z1–Z5 delivery zones) and 17 edges. Each edge is labelled with distance in km.

Adjacency List Representation:

Node	Neighbors (distance in km)
S1	Z1 (4 km), Z2 (3 km), S2 (5 km), S3 (9 km)
S2	S1 (5 km), Z2 (2 km), Z1 (6 km), S3 (7 km)
S3	S2 (7 km), Z3 (3 km), S4 (8 km), Z4 (6 km), S1 (9 km)
S4	S3 (8 km), Z4 (3 km), S5 (10 km)
S5	S4 (10 km), Z5 (4 km)
Z1	S1 (4 km), S2 (6 km), Z3 (6 km)

Z2	S1 (3 km), S2 (2 km), Z3 (5 km)
Z3	S3 (3 km), Z1 (6 km), Z2 (5 km), Z4 (7 km)
Z4	S3 (6 km), S4 (3 km), Z3 (7 km), Z5 (9 km)
Z5	S5 (4 km), Z4 (9 km)

Part B — BFS Traversal (starting from S3)

Tie-breaking rule: S nodes before Z nodes; lower numbers first. BFS uses a queue (FIFO).

Step	Visit	Queue After Processing	Visited So Far
1	S3	[S1, S2, S4, Z3, Z4]	S3
2	S1	[S2, S4, Z3, Z4, Z1, Z2]	S3, S1
3	S2	[S4, Z3, Z4, Z1, Z2]	S3, S1, S2
4	S4	[Z3, Z4, Z1, Z2, S5]	S3, S1, S2, S4
5	Z3	[Z4, Z1, Z2, S5]	S3, S1, S2, S4, Z3
6	Z4	[Z1, Z2, S5, Z5]	S3, S1, S2, S4, Z3, Z4
7	Z1	[Z2, S5, Z5]	S3, S1, S2, S4, Z3, Z4, Z1
8	Z2	[S5, Z5]	S3, S1, S2, S4, Z3, Z4, Z1, Z2
9	S5	[Z5]	S3, S1, S2, S4, Z3, Z4, Z1, Z2, S5
10	Z5	[]	S3, S1, S2, S4, Z3, Z4, Z1, Z2, S5, Z5

BFS Visit Order: S3 → S1 → S2 → S4 → Z3 → Z4 → Z1 → Z2 → S5 → Z5

Explanation: S3's neighbours in sorted order are S1, S2, S4, Z3, Z4 — all added to the queue. S1 is dequeued next; its unvisited neighbours Z1, Z2 are added. S2 is dequeued; no new neighbours. S4 adds S5. Z3, Z4 are processed without new additions until Z4 adds Z5.

Part C — DFS Traversal (starting from S3)

DFS uses a recursive call stack (or explicit stack). At each node, unvisited neighbours are explored in sorted order (S before Z, lower first).

Step	Visit	Call Stack State
1	S3	[S3]
2	S1	[S3, S1] — S1 is first sorted unvisited neighbour of S3
3	S2	[S3, S1, S2] — S2 is first sorted unvisited neighbour of S1
4	Z1	[S3, S1, S2, Z1] — Z1 is first sorted unvisited neighbour of S2
5	Z3	[S3, S1, S2, Z1, Z3] — Z3 is first unvisited neighbour of Z1

6	Z2	[S3, S1, S2, Z1, Z3, Z2] — Z2 is unvisited neighbour of Z3
7	Z4	[S3, S1, S2, Z1, Z3, Z4] — Z4 is next unvisited neighbour of Z3
8	S4	[S3, S1, S2, Z1, Z3, Z4, S4] — S4 is unvisited neighbour of Z4
9	S5	[S3, S1, S2, Z1, Z3, Z4, S4, S5]
10	Z5	[S3, S1, S2, Z1, Z3, Z4, S4, S5, Z5] — all nodes visited

DFS Visit Order: S3 → S1 → S2 → Z1 → Z3 → Z2 → Z4 → S4 → S5 → Z5

Part D — Business Application: Urgent Order at Z5

Using BFS reasoning from Z5, counting hops to each dark store:

Dark Store	Hops from Z5	Path
S5	1 hop	Z5 → S5
S4	2 hops	Z5 → S5 → S4 (or Z5 → Z4 → S4)
S3	2 hops	Z5 → Z4 → S3
S1	3 hops	Z5 → S5 → S4 → S3 → S1
S2	3 hops	Z5 → S5 → S4 → S3 → S2

Recommendation: FreshBasket should dispatch from S5 (Silicon Oasis Dark Store). It is only 1 hop away from Z5 (International City), connected by a direct 4 km road with an 8-minute travel time. This is the shortest path both by hops and by actual distance, ensuring the fastest delivery for the urgent order.

Problem 2: Shortest Delivery Routes

Topics: Dijkstra's Algorithm, Floyd-Warshall

Part A — Dijkstra's Algorithm: S2 to Z4

Starting node: S2 (distance = 0). All other nodes initialised to ∞. At each step, the unvisited node with the minimum tentative distance is selected and its neighbours are relaxed.

Step	Visit	S1	S2	S3	S4	S5	Z1	Z2	Z3	Z4	Z5
Init	—	∞	0	∞	∞	∞	∞	∞	∞	∞	∞
1	S2	5	0	7	∞	∞	6	2	∞	∞	∞
2	Z2	5	0	7	∞	∞	6	2	7	∞	∞
3	S1	5	0	7	∞	∞	6	2	7	∞	∞
4	Z1	5	0	7	∞	∞	6	2	7	∞	∞

5	S3	5	0	7	15	∞	6	2	7	13	∞
6	Z3	5	0	7	15	∞	6	2	7	13	∞
7	Z4	5	0	7	15	∞	6	2	7	13	22
8	S4	5	0	7	15	25	6	2	7	13	22
9	Z5	5	0	7	15	25	6	2	7	13	22
10	S5	5	0	7	15	25	6	2	7	13	22

Shortest path from S2 to Z4: S2 → S3 → Z4

Total distance: 13 km | Total cost: AED 9.00 (S2→S3: AED 5.00 + S3→Z4: AED 4.00)

Step-by-step logic: S2 is dequeued first (dist=0). Neighbours S1(5), Z2(2), Z1(6), S3(7) are updated. Z2 is dequeued (dist=2). It updates Z3 to 7. S1 is next (dist=5), no improvements. Z1 (dist=6), no improvements. S3 is dequeued (dist=7); it updates S4 to 15 and Z4 to 13. Z3 (dist=7), no improvements. Z4 is reached with cost 13 km.

Part B — Floyd-Warshall: Dark Stores Only (S1–S5)

Only direct edges between dark stores are used in the initial matrix. Delivery zones are excluded since we are computing inter-store distances for stock transfers.

Direct edges between dark stores: S1–S2 (5 km), S2–S3 (7 km), S3–S4 (8 km), S4–S5 (10 km), S1–S3 (9 km). No direct edges between S1–S4, S1–S5, S2–S4, S2–S5, S3–S5.

Initial Distance Matrix:

	S1	S2	S3	S4	S5
S1	0	5	9	∞	∞
S2	5	0	7	∞	∞
S3	9	7	0	8	∞
S4	∞	∞	8	0	10
S5	∞	∞	∞	10	0

Floyd-Warshall iterates through each node k as a potential intermediate. After k=S1 and k=S2: no improvements (S1, S2 do not bridge any disconnected pairs). After k=S3: S1↔S4 improves to 9+8=17; S2↔S4 improves to 7+8=15; S1↔S5 and S2↔S5 remain ∞ until S4 is used. After k=S4: S1↔S5 = 17+10=27; S2↔S5 = 15+10=25; S3↔S5 = 8+10=18.

Final Distance Matrix:

	S1	S2	S3	S4	S5
S1	0	5	9	17	27
S2	5	0	7	15	25
S3	9	7	0	8	18

S4	17	15	8	0	10
S5	27	25	18	10	0

Cells highlighted in red represent the worst-case inter-store distance.

Analysis:

- Farthest pair: S1 and S5 — distance of 27 km (path: S1→S3→S4→S5)
- Best-connected store (lowest average distance): S3 — average distance = (9+7+0+8+18)/4 = 10.5 km, the lowest among all stores
- Recommended new road: S1–S4 direct link. Currently S1↔S5 = 27 km and S1↔S4 = 17 km (via S3). A direct S1–S4 road of ~11 km would reduce S1↔S4 to 11 km and S1↔S5 to 21 km, cutting the worst-case by 6 km and improving network resilience across the Al Barsha–Deira corridor.

Problem 3: Order Management Pipeline

Topics: BST, Linked List, Priority Queue, Stack

Part A — BST for Order Lookup

1. BST Construction (Insertion sequence: 5042, 5018, 5067, 5009, 5031, 5055, 5081, 5025)

Each value is inserted by comparing with the current node: smaller → go left, larger → go right.

```
          5042  (root)
         /    \
     5018     5067
    /    \    /    \
5009  5031 5055  5081
      /
    5025
```

Insertion trace: 5042→root; 5018<5042→left; 5067>5042→right; 5009<5042→left, <5018→left of 5018; 5031<5042→left, >5018→right of 5018; 5055>5042→right, <5067→left of 5067; 5081>5042→right, >5067→right of 5067; 5025<5042→left, >5018→right of 5018, <5031→left of 5031.

2. Tree Traversals:

- In-order (L→Root→R): 5009, 5018, 5025, 5031, 5042, 5055, 5067, 5081 [sorted ascending — key BST property]
- Pre-order (Root→L→R): 5042, 5018, 5009, 5031, 5025, 5067, 5055, 5081
- Post-order (L→R→Root): 5009, 5025, 5031, 5018, 5055, 5081, 5067, 5042

3. Search trace for Order 5055:

Step	Current Node	Comparison	Direction
1	5042	5055 > 5042	Go Right
2	5067	5055 < 5067	Go Left
3	5055	5055 = 5055	FOUND

Number of comparisons: 3

4. Deletion of Order 5067 — Case: Two Children

5067 has two children (5055 and 5081). The deletion rule for two children: find the in-order successor (the smallest value in the right subtree). The in-order successor of 5067 is 5081 (it has no left child in the right subtree). Replace 5067's key with 5081, then delete 5081 from its original position (a leaf deletion).

BST after deleting 5067:

```
                 5042  (root)
                /    \
            5018     5081  ← replaced 5067
           /    \    /
       5009  5031 5055
             /
           5025
```

Part B — Delivery Route as a Doubly Linked List

Original route:

```
NULL ⇄ [Z1] ⇄ [Z3] ⇄ [Z2] ⇄ [Z4] ⇄ [Z5] ⇄ NULL
```

1. Insert Z3 at head (VIP order) — Remove duplicate:

Decision: Remove the duplicate Z3 from the middle of the list. Visiting Z3 twice wastes time and fuel. The VIP order at Z3 needs to be prioritised, so Z3 should appear only once — at the head.

```
NULL ⇄ [Z3] ⇄ [Z1] ⇄ [Z2] ⇄ [Z4] ⇄ [Z5] ⇄ NULL
```

Pointer operations: new head's prev=NULL, new head's next=Z1, Z1's prev=Z3, old Z3 node's prev/next pointers updated to bridge around it (Z2.next=Z4, Z4.prev=Z2).

2. Remove Z4 (order cancelled):

```
NULL ⇄ [Z3] ⇄ [Z1] ⇄ [Z2] ⇄ [Z5] ⇄ NULL
```

Pointer operations: Z2.next = Z5; Z5.prev = Z2. Z4's node pointers are cleared.

3. Why a doubly linked list is better than an array for rider routes:

A doubly linked list allows O(1) insertions and deletions at any position once the target node is found — ideal for dynamic route changes (VIP insertions, cancellations). Arrays require O(n) shifting for middle insertions/deletions. The doubly-linked structure also supports backward

traversal, useful if the rider needs to back-track. Arrays offer O(1) random access but rider routes are traversed sequentially anyway, making this advantage irrelevant here.

Part C — Priority Queue & Stack

1. Min-Heap after inserting all 6 orders (A:3, B:1, C:4, D:1, E:2, F:5):

Insertion and heapify-up trace: A(3) inserted; B(1) inserted, swaps up past A(3) to root; C(4) inserted, stays as right child; D(1) inserted, swaps up past A(3), reaches level 1 equal to B(1) — no further swap (ties do not swap in standard min-heap); E(2) inserted, parent is D(1) > E(2) — no swap needed (D=1 < E=2, so no swap); F(5) inserted, parent is C(4) < F(5) — no swap.

Heap as binary tree (array representation: B, D, C, A, E, F):

```
          B(1)          ← index 0
         /    \
      D(1)    C(4)       ← index 1, 2
      / \     /
  A(3) E(2) F(5)        ← index 3, 4, 5
```

2. Dequeue all 6 orders (min-heap extract-min with heapify-down):

Dequeue #	Order	Priority	Heap State After Extraction
1st	B	1	[D(1), A(3), C(4), F(5), E(2)] → after heapify: [D(1), A(3), C(4), F(5), E(2)]
2nd	D	1	[E(2), A(3), C(4), F(5)] → after heapify: [E(2), A(3), C(4), F(5)]
3rd	E	2	[A(3), F(5), C(4)] → after heapify: [A(3), F(5), C(4)]
4th	A	3	[C(4), F(5)] → after heapify: [C(4), F(5)]
5th	C	4	[F(5)]
6th	F	5	[]

Dequeue order: B, D, E, A, C, F

FIFO guarantee for equal-priority orders: A standard min-heap does NOT guarantee FIFO ordering for orders with equal priority. B and D are both Priority 1, but the heap's internal restructuring during heapify operations can place either at the root — the order depends on the heap's current shape, not arrival time. To enforce FIFO for equal-priority orders, add a secondary sort key (a sequence/timestamp counter) to each entry: compare (priority, arrival_time). Use a tuple comparison so ties are broken by arrival order. This ensures that among Priority-1 orders, B (arrived 2nd) is always dequeued before D (arrived 4th).

3. Stack — Order B status with Undo:

Status history pushed onto a stack: Received → Confirmed → Preparing → Dispatched

Stack before undo (top on right):

```
[Received | Confirmed | Preparing | Dispatched]  ← TOP
```

After pop (undo 'Dispatched'):

```
[Received | Confirmed | Preparing]  ← TOP
```

Order B's current status: Preparing. The stack correctly models the undo operation; the dispatcher has rolled back the premature 'Dispatched' state.

Problem 4: Sorting, Searching & Infrastructure

Topics: Merge Sort, Binary Search, MST (Kruskal's), Divide and Conquer

Part A — Merge Sort

Unsorted: [12, 3, 7, 9, 2, 15, 6, 4]

Divide Phase (splitting into single elements):

```
Level 0:  [12, 3, 7, 9, 2, 15, 6, 4]
Level 1:  [12, 3, 7, 9]        [2, 15, 6, 4]
Level 2:  [12, 3]  [7, 9]      [2, 15]  [6, 4]
Level 3:  [12][3]  [7][9]      [2][15]  [6][4]
```

Merge Phase (with comparison counts):

Level	Left	Right	Merged Result	Comparisons
3	[12]	[3]	[3, 12]	1
3	[7]	[9]	[7, 9]	1
3	[2]	[15]	[2, 15]	1
3	[6]	[4]	[4, 6]	1
2	[3, 12]	[7, 9]	[3, 7, 9, 12]	3
2	[2, 15]	[4, 6]	[2, 4, 6, 15]	3
1	[3, 7, 9, 12]	[2, 4, 6, 15]	[2, 3, 4, 6, 7, 9, 12, 15]	7

Sorted result: [2, 3, 4, 6, 7, 9, 12, 15]

Total comparisons in merge phase: 1+1+1+1+3+3+7 = 17 comparisons

Time complexity of Merge Sort: O(n log n) — always, regardless of input order.

Part B — Binary Search

Array: [2, 3, 4, 6, 7, 9, 12, 15] (indices 0–7) | Target: 9

Step	Low	High	Mid	arr[Mid]	Comparison	Action
1	0	7	3	6	9 > 6	Search right → low=4
2	4	7	5	9	9 = 9	FOUND at index 5

Found at index 5. Binary search required only 2 comparisons.
Linear search for value 9: 9 is at index 5, requiring 6 comparisons (check index 0,1,2,3,4,5).

Scale analysis — 500 customer lookups on a sorted list of 1,000 orders:

Method	Comparisons per lookup	Total daily comparisons
Binary Search	$\lceil \log_2(1000) \rceil = 10$	500 × 10 = 5,000
Linear Search (avg)	500 (average case)	500 × 500 = 250,000
Linear Search (worst)	1,000 (worst case)	500 × 1,000 = 500,000

Binary search performs 5,000 comparisons vs 250,000 (average) for linear — a 50× improvement on average and 100× improvement in the worst case. This directly reduces server CPU time and enables FreshBasket to handle the tripled Ramadan order volume.

Part C — Minimum Spanning Tree (Kruskal's Algorithm)

Kruskal's algorithm adds edges in ascending cost order, rejecting any edge that would create a cycle. We use a Union-Find (disjoint set) data structure to detect cycles. We need 9 edges to connect all 10 nodes.

Step 1 — Edges sorted by cost (AED):

#	Edge	Cost (AED)	Decision	Reason	Count
1	S2–Z2	1.5	ADD	No cycle — connects disjoint components {S2} and {Z2}	1
2	S1–Z2	2.0	ADD	No cycle — connects {S1} to {S2,Z2}	2
3	S3–Z3	2.0	ADD	No cycle — connects {S3} and {Z3}	3
4	S4–Z4	2.0	ADD	No cycle — connects {S4} and {Z4}	4
5	S1–Z1	2.5	ADD	No cycle — connects {Z1} to {S1,S2,Z2}	5
6	S5–Z5	2.5	ADD	No cycle — connects {S5} and {Z5}	6
7	S1–S2	3.5	REJECT	Cycle — S1 and S2 already in the same component	6
8	Z2–Z3	3.5	ADD	No cycle — connects {S1,S2,Z2,Z1} to {S3,Z3}	7
9	S2–Z1	4.0	REJECT	Cycle — S2 and Z1 already connected	7

10	S3–Z4	4.0	ADD	No cycle — connects {S3,Z3,S1,S2,Z2,Z1} to {S4,Z4}	8
11	Z1–Z3	4.0	REJECT	Cycle — Z1 and Z3 now in same component	8
12	S2–S3	5.0	REJECT	Cycle — both in same component	8
13	Z3–Z4	5.0	REJECT	Cycle — both in same component	8
14	S3–S4	5.5	REJECT	Cycle — both in same component	8
15	Z4–Z5	6.0	ADD	No cycle — connects {S5,Z5} to main component	9

Green rows = edges added to MST. Red rows = rejected edges (would form a cycle).

Total MST cost: AED 26.00

MST edges: S2–Z2, S1–Z2, S3–Z3, S4–Z4, S1–Z1, S5–Z5, Z2–Z3, S3–Z4, Z4–Z5

Part D — Divide and Conquer for Peak Delivery Hour

1. Divide-and-Conquer Approach:

- Divide: Split the 24-hour array in half at the midpoint. Recursively divide each half until subarrays of size 1 are reached (base case).
- Conquer: A single-element subarray is trivially the peak of itself — return it directly.
- Combine: Compare the peak of the left half against the peak of the right half. Return the one with the higher order count. This comparison propagates up, ensuring each level correctly identifies the peak.

2. Complexity Analysis:

Divide-and-conquer: $T(n) = 2T(n/2) + O(1) \rightarrow O(n)$ by the Master Theorem.

Linear scan: $O(n)$ — iterate once, track running maximum.

Honest assessment: For this specific problem (finding a single maximum in an unsorted array), divide-and-conquer is NOT better than a linear scan. Both are $O(n)$. The linear scan is simpler to implement, uses $O(1)$ extra space, and has better cache locality with no recursive call overhead. Divide-and-conquer becomes advantageous when: (a) finding multiple peaks, (b) the problem has overlapping subproblems (then memoisation helps), or (c) parallelism is available to process halves simultaneously. For FreshBasket's single-peak problem, the linear scan is the pragmatically superior choice.

Layer 2: Case Study — The Ramadan Surge

Scenario: Ramadan order volume triples from 800 to 2,400/day for 30 days. Budget: AED 50,000. Timeline: 5 days. Three competing problems must be solved within these constraints.

CS-1: Prioritization Decision

Priority Summary Table:

Priority	Problem	Cost (AED)	Time	Monthly Impact
1st	Problem Y — Order Pile-Up (Priority Queue)	15,000	2 days	AED 80,000/month (VIP retention)
2nd	Problem X — Route Chaos (Dijkstra's)	25,000	4 days	AED 35,000/month savings
Dropped	Problem Z — Dark Store Connectivity (MST)	30,000	5 days	AED 20,000/month (stock-out reduction)

Justification (300–400 words):

Problem Y (Priority Queue) is the clear first priority on both financial and operational grounds. The ROI is exceptional: AED 15,000 investment protects AED 80,000/month in revenue, a 5.3× monthly return starting in Week 1. VIPs represent 35% of revenue, and 15% churn among this group would translate to permanent, compounding revenue loss — not just a one-month problem. Crucially, Problem Y can be built in 2 days, meaning it can be completed and tested before Ramadan begins. A priority queue (min-heap) is a well-understood structure that any competent developer can implement and validate quickly. The risk of shipping a buggy dispatch system during a 3× volume event is lower here than with the other problems.

Problem X (Route Chaos) ranks second because AED 35,000/month in fuel savings compounds across every Ramadan and beyond. During Iftar peak (5:00–7:30 PM), the 30% excess distance translates directly into late deliveries and brand damage. We implement Floyd-Warshall pre-computation for all-pairs shortest paths across the 10-node network — a one-time overnight computation that all 40 riders' dispatch decisions draw from in O(1) lookup time. The 4-day build time is tight but workable because Problem Y is done in 2 days, freeing the team. Combined budget: AED 40,000, within the AED 50,000 limit.

Problem Z (Dark Store Connectivity) is dropped despite its operational value. The AED 30,000 cost would exceed the combined budget with either alternative. More critically, 5 days of build time exceeds the available timeline: if the team is also building Problem Y (2 days) and Problem X (4 days) in parallel, there is no engineering capacity for a 5-day infrastructure project. The business risk of deferring Problem Z is manageable: stock-outs can still be resolved manually (via the existing WhatsApp process) during Ramadan. The 40% stock-out reduction is valuable but not existential — VIP churn is existential. Problem Z should be scheduled as a post-Ramadan initiative once revenue is protected and engineering bandwidth is restored.

Dependency note: Problem Y and Problem X are independent — the priority queue manages dispatch ordering while Dijkstra's determines route paths. No blocking dependency exists. Problem Z would benefit from Problem X's routing data (optimal transfer paths), providing an additional reason to implement X before Z later.

CS-2: Algorithm Selection for Priority 1 (Problem Y)

Choice: Min-Heap Priority Queue

We implement a binary min-heap where each order is keyed by (priority_level, arrival_timestamp). Priority 1 is VIP 30-minute, Priority 5 is Economy. The min-heap always surfaces the highest-urgency, earliest-arriving order at the root, enabling O(log n) insertion and O(log n) extraction.

Why not a simple FIFO queue? FIFO treats all orders equally regardless of revenue value. During Ramadan peak (2,400 orders/day ≈ 100 orders/hour), FIFO creates 45-minute VIP wait times — unacceptable given 15% projected churn.

Why not multiple FIFO queues per zone? Zone-based FIFO improves routing locality but still does not prioritise VIPs over economy customers within a zone. A VIP in Z3 would wait behind 20 economy orders that arrived first. This does not solve the core problem.

Why not BST-based indexing? A BST provides O(log n) insertion and O(log n) min extraction, similar to a heap. However, heap operations have smaller constants because they use array-based storage with O(1) index arithmetic and excellent cache locality. A BST requires pointer manipulation and has higher memory overhead — disadvantages under high load.

Time complexity: O(log n) per insertion and extraction, where n = queue size. At peak (100 orders/hour), n ≤ 200 at any time, so $\log_2(200) \approx 8$ comparisons — negligible latency.

Assumptions: Priority levels are correctly set by the frontend before enqueueing. Risks: If the priority assignment logic has bugs, VIP orders may be mis-tagged and land in the economy queue. Additionally, during extreme spikes (Iftar burst), the 40 riders may not have enough capacity regardless of dispatch ordering — the heap optimises who waits less, not how many riders are available.

CS-3: Trade-Off Analysis

Factor	Problem Y — Min-Heap Priority Queue	Problem X — Floyd-Warshall Route Pre-computation
Time complexity	O(log n) insert/extract	$O(n^3)$ pre-compute; O(1) lookup at runtime
Space complexity	O(n) for heap array	$O(n^2)$ for distance matrix; n=10 → 100 cells
Implementation time	2 days	4 days
Failure mode	Wrong priority tags → VIPs mis-routed; heap corruption under concurrent writes	Stale pre-computed matrix if road network changes; no real-time traffic
What you sacrifice	FIFO fairness for economy customers; complexity vs simple queue	Real-time adaptability; does not handle unexpected road closures

What you gain	VIP SLA compliance; AED 80,000/month protection; 15% churn avoided	30% distance reduction; AED 35,000/month fuel savings; scalable to any graph size

Biggest risk and fallback:

The biggest risk is that Problem X is not deployable in time: if the route optimization system has bugs at go-live, riders will revert to manual route selection, erasing the expected fuel savings. Fallback plan for Week 1: pre-compute the Floyd-Warshall matrix offline before Ramadan and provide it as a printed reference card for each rider's top 3 delivery zones. This low-tech fallback delivers ~60% of the efficiency gain (most deliveries are from the same 2–3 store-zone pairs) while the software is debugged. Problem Y's fallback is simpler: revert to dual FIFO queues (VIP and standard), which is manually manageable and preserves most of the VIP benefit.

CS-4: Stakeholder Communication

1. For the CEO (Non-technical, budget approval tonight):

We have identified the two highest-impact fixes for Ramadan. First, we will overhaul our order dispatch system so VIP customers — who generate 35% of our revenue — are always served first. This AED 15,000 upgrade protects an estimated AED 80,000 per month in VIP revenue and prevents a projected 15% VIP churn. Second, we will optimise rider routes to eliminate the 30% excess distance during Iftar peak, saving AED 35,000 per month in fuel. Total investment: AED 40,000. Both systems will be live before Ramadan begins. The dark store connectivity project will follow post-Ramadan.

2. For the Lead Engineer (Technical, starts coding tomorrow):

Day 1–2: Build a binary min-heap priority queue with composite key (priority_level, unix_timestamp) for FIFO tiebreaking among equal-priority orders. O(log n) insert and extract-min. Thread-safe implementation required — use a mutex or concurrent priority queue. Unit-test edge cases: equal priorities, empty queue, single-element. Day 2–4 (parallel): Run Floyd-Warshall on the 10-node graph ($O(n^3) = 1000$ ops, trivial). Store the 10×10 distance matrix in Redis for O(1) dispatch lookups. Rebuild matrix nightly. No real-time traffic updates for now — static weights are sufficient. Start with Problem Y since it has the larger revenue impact and shorter build time.

Layer 3: Critical Thinking

CT-1: The 'Just Use Google Maps API' Challenge

Response to the engineering meeting (300–400 words):

The senior engineer makes a valid point that deserves a direct response, not dismissal. Google

Maps API is mature, globally maintained, handles real-time traffic, and would be faster to integrate than building Dijkstra's from scratch. For a startup team under a 5-day deadline, the API is genuinely compelling.

However, there are at least three meaningful reasons why in-house algorithmic capability benefits FreshBasket at scale:

1. Cost at scale: Google Maps API charges per request. At 2,400 orders/day × 2 route calculations (planning + re-route) × 30 Ramadan days, FreshBasket could face thousands of API calls daily. At Google's standard pricing (approximately USD 0.005 per directions request), this is manageable now — but at 10,000 orders/day in two years, API costs become a meaningful operational expense. A pre-computed Floyd-Warshall matrix on a 10-node graph is computed once and costs nothing at query time.

2. Customisation and multi-stop optimisation: Google Maps optimises routes for general traffic. FreshBasket's dispatch problem is not simply A-to-B routing — it involves multi-stop delivery sequencing across rider zones, priority-weighted stops (VIP first), and dark-store assignment logic. The API does not natively handle the joint problem of 'which store dispatches which rider to which zone in which order.' In-house code can integrate these constraints directly into the routing algorithm.

3. Data privacy and competitive moat: Every route query to Google reveals FreshBasket's delivery patterns — which zones are high-demand, which dark stores are most active, and at what times. This data, if aggregated by Google or subpoenaed, could eventually inform a competitor. An in-house system keeps operational intelligence proprietary. Additionally, the engineering capability to model and optimise a logistics network is a genuine competitive moat that cannot be commoditised.

Scenario where Google Maps API is genuinely better: During an unexpected road closure on Sheikh Zayed Road mid-shift, real-time rerouting is needed immediately. A pre-computed static matrix cannot reflect live traffic. Here, the API's real-time traffic layer is unambiguously superior.

Concrete recommendation: Implement a hybrid. Use the in-house Floyd-Warshall pre-computed matrix as the primary dispatch engine for all planned routes (O(1) lookup, zero API cost, fully customisable). Use the Google Maps API only for real-time re-routing exceptions — triggered when a rider is stationary for more than 5 minutes or a traffic incident is detected. This approach captures the customisation and cost benefits of in-house development while retaining the API's real-time advantage for edge cases, at a fraction of the full-API cost.

CT-2: Data Structures Are a Team Decision

1. Would you implement an AVL tree? (200–250 words):

No. The textbook answer is theoretically correct, but this team cannot safely ship an AVL tree under Ramadan time pressure.

AVL tree implementation requires handling four rotation cases (LL, RR, LR, RL), maintaining balance factors at every node, and correctly propagating rebalancing up the tree on every insertion and deletion. For a senior developer who 'prefers simple solutions,' this is already a stretch. For two junior developers fresh from university, the debugging burden alone could consume the entire 5-day window. A bug in the rotation logic during Ramadan peak — where 2,400 orders per day are flowing through the system — is a production disaster with no clean rollback path.

The 80% solution: Use Python's sortedcontainers.SortedList, which provides O(log n) lookups and insertions backed by a skip-list under the hood. It is battle-tested, installable in one pip command, and requires zero tree-rotation knowledge. The senior developer can explain it to the juniors in an afternoon. The data analyst can even write SQL-equivalent pseudocode to verify the ordering. The performance characteristics are nearly identical to AVL for n < 100,000 — well beyond FreshBasket's current order volume. Save the AVL tree for a future sprint when there is time to do it properly.

2. Evaluation of the Python dictionary + sorted list suggestion:

The suggestion is good enough for now, and the senior developer deserves credit for prioritising shippability over theoretical elegance. A Python dict provides O(1) average-case lookup by Order ID — faster than any tree for direct access. A sorted list provides O(log n) binary search for range queries. For the current volume (2,400 orders/day, peaking at ~200 in the queue simultaneously), this is entirely sufficient.

At what scale does it break? Python dicts scale well to millions of entries, so the dict is not the bottleneck. The sorted list becomes expensive at O(n) insertions as the list grows beyond ~50,000 items, because inserting into the middle of an array requires shifting. If FreshBasket scales to 50,000 concurrent orders (a plausible Year 3 scenario), a tree-based structure with O(log n) guaranteed insertion becomes necessary. The transition point is clear: migrate to a proper BST or database index when insertion latency becomes measurable.

3. Slack message to the team:

Hey team — wanted to share the final call on the order lookup system. After thinking through the options carefully, we're going with a Python dict for direct Order ID lookups combined with sortedcontainers.SortedList for priority-based ordering. This gives us O(1) direct access and O(log n) sorted queries — essentially the same performance as an AVL tree for our current scale, without the implementation complexity. Huge respect to [senior dev] for pushing the pragmatic approach here; you were right that shippability matters more than theoretical optimality right now. For the junior devs: you'll be working with well-documented, widely-used libraries, which means great resources and no debugging rabbit holes. We can revisit a tree-based structure in Q3 when we have breathing room. Let's build something reliable we're proud to ship. See everyone at standup tomorrow.

CT-3: When the Algorithm Is Right but the Business Is Wrong

1. Memo to the CEO — S5 Silicon Oasis Recommendation:

MEMORANDUM

TO: Chief Executive Officer, FreshBasket UAE

FROM: Analytics & Engineering Team

RE: Silicon Oasis Dark Store (S5) — Closure Recommendation Review

DATE: April 2026

Our Floyd-Warshall network analysis confirms that S5 is the most poorly connected dark store, with an average inter-store distance of 20 km — nearly double S3's 10.5 km. The algorithm correctly identifies that closing S5 and redirecting its 120 daily orders to S4 would reduce network-wide delivery costs by approximately 12%.

We recommend against closing S5. The recommendation rests on three business realities the algorithm cannot see. First, International City (Z5) hosts one of our fastest-growing customer segments: 40% month-over-month growth among the expat community. Closing S5 extends delivery time from 8 minutes to 38 minutes — a 375% increase that is incompatible with our 30-minute promise and sufficient to drive this segment permanently to a competitor. Second, the 12% cost reduction (approximately AED 20,000/month at current scale) is dwarfed by the revenue potential of a segment growing at 40% MoM. At this growth rate, Z5 could represent 15–20% of total order volume within six months. Third, Noon Minutes is planning a Silicon Oasis dark store next quarter. Closing S5 would concede this territory at precisely the moment we should be entrenching our position.

Proposed third option: Rather than closing S5, investigate increasing its throughput. The algorithm's cost concern is driven by S5's isolation, not its existence. A dedicated direct logistics link between S4 and S5 (reducing the 10 km gap) and improved stock replenishment frequency would reduce isolation costs while retaining the Z5 customer base.

Additional data needed before a confident decision: (1) Customer lifetime value segmentation for Z5 users vs. the Dubai average; (2) Actual fuel and transfer cost breakdown for S5's 120 daily orders; (3) Noon Minutes' confirmed timeline and store location in Silicon Oasis; (4) Delivery time elasticity data — at what delivery time do Z5 customers churn?

2. Reflection — When should a manager override an algorithm's recommendation?

A manager should override an algorithmic recommendation when the model's objective function does not capture the full business reality — as is the case here, where the algorithm optimises current costs but ignores growth trajectories, competitive dynamics, and brand commitments. The overriding principle should be: algorithms are optimal within the constraints and objectives they are given, but defining those constraints and objectives correctly is a human responsibility. Managers should override when (a) the algorithm's inputs are outdated or incomplete, (b) the optimisation target is the wrong target, or (c) strategic factors — customer loyalty, competitive response, brand trust — are unquantifiable but material. The override should always be documented with explicit reasoning, so it can be revisited when new data arrives.